

GAME NAME

Game Design Document (template)

Copyright Information

Version history

Time	Author	Changes

Game Overview

Game Concept

Shoki is an alien whose spacecraft has crash-landed on Earth. Government human forces are closing in to capture him. The player must survive 10 escalating waves of human soldiers on an 8×8 isometric grid, repair the ship between rounds, and escape home.

The game blends the readable telegraph system of Into the Breach with the build-craft progression of a roguelite. Every turn is a small puzzle: where to move, melee or ranged, bait into a trap or avoid it, save HP for later.

Feature Set

- Turn-based tactical combat on an isometric 8×8 grid
- Initiative-based turn order (d20 + speed)
- 3 enemy archetypes — Soldier (melee grunt), Sniper (ranged), Heavy (high-HP melee)
- 3 recruitable companions — Drone (glass cannon), Brawler (tank), Trickster (acts twice)
- Enemy telegraph system — every attack and move shown one turn in advance
- Environmental traps that affect both player and enemies
- Permadeath companions + damage-multiplier upgrades stacked across the run
- Procedural room layout regenerated each wave

Genre

Turn-based tactical roguelite.

Target Audience

Players who enjoy Into the Breach, Mewgenics, FTL, and Shovel Knight — readable systems, high replayability, short runs (~20 minutes).

Game Flow

Micro (one turn): Move $\leq$ moveRange tiles → trap check on arrival → optional attack → end turn → enemies execute telegraphed plans.

Macro (one wave): Survive all enemies → 75% of wave damage heals → +25% melee or ranged upgrade → at waves 3, 6, 9 choose a companion → new room generates.
Meta (one run): Survive all 10 waves. HP carries across waves; damage multipliers compound; companion roster grows.

Look and Feel

2D pixel art, low-resolution (16-pixel base unit). Inspired by Shovel Knight (palette + chunky pixels), Into the Breach (telegraph clarity), Mewgenics (creature design). Isometric 2:1 diamond projection with tiles that have a 3D-cube appearance.
LeBlanc fun type — Challenge: every turn is a readable but non-trivial decision. No hidden information; outcome rests on the player reading the board correctly.

Number of locations

One — a single crash-site room, procedurally regenerated each wave with new void cluster shapes, obstacles, traps, and spawn points.

Number of levels

10 waves. Each wave is a complete combat encounter; the room regenerates between waves.

Number of NPC’s

Number of weapons

Two attack modes per character (melee and ranged) toggleable each turn. Damage values are character-specific and modified by per-run +25% upgrades.

Gameplay and Mechanics

Gameplay

Each turn the player moves up to moveRange tiles, may trigger a trap on arrival, and chooses one attack (melee adjacent or ranged 2–4 tiles). Enemies acted in initiative order, committing to their telegraphed plans (yellow tile = attack target, grey = move target). The player can outplay the AI by reading the telegraphs and stepping off attack tiles or baiting enemies onto traps.

Game Progression

Reused from Gameplay section — the 10-wave structure carries the entire arc, no cutscenes required. Difficulty scales over 10 waves through enemy composition. Soldier counts grow modestly; Sniper introduced at wave 4, Heavy at wave 6. HP carries across waves with a 75% post-wave heal. Damage multipliers from upgrades compound for the full run.

Wave	Soldiers	Snipers	Heavies	Notes
1	2	0	0	Onboarding — pure melee rush
2	2	0	0	Same composition, learning curve
3	3	0	0	First Companion pick
4	2	1	0	Sniper introduced — ranged threat
5	2	1	0	Practice mixed composition
6	2	1	1	Heavy introduced + second Companion pick
7	2	2	0	Double sniper — positioning matters
8	1	2	1	Hardest single composition
9	2	1	1	Third Companion pick
10	2	1	2	Boss-feel finale — two heavies

Mission/challenge Structure

Each wave is a self-contained combat encounter. Win condition: defeat all enemies on the board. There are no sub-objectives — just survive.

Puzzle Structure

Each turn is a micro-puzzle in itself: the telegraph system shows next-turn enemy actions, so the player solves "where can I stand to avoid damage AND get an attack off?" That's the core minute-to-minute puzzle.

Objectives

Win condition: Survive all 10 waves.

Lose condition: Shoki's HP reaches 0 (companion losses do not end the run).

Play Flow

MainMenu → Combat (wave 1) → UpgradeScreen → [CompanionScreen at waves 3/6/9] → Combat (wave N+1) → ... → Win or GameOver. Restart returns to MainMenu.

Mechanics

Every mechanic listed below maps to a class in the codebase (GridManager, TurnManager, CombatSystem, TelegraphSystem, TrapSystem, etc.) — see the Technology section for the system overview.

Physics

None — grid-based discrete movement. No simulated physics, no momentum, no continuous collision. Tile occupancy is binary.

Movement

Cardinal-only pathfinding via 4-direction BFS. Player moveRange = 2 tiles. Move highlights show only tiles reachable in those steps via the same BFS — so highlights always match what the walker can actually reach. Movement plays as a smooth ~0.16-second tile-to-tile lerp; traps fire on visual arrival.

Objects

Spike trap: 10 damage on entry, then consumed (tile reverts to ground).

Pit trap: Instant kill (9999 damage); does not consume.

Slow zone: Applies −1 moveRange for one turn; does not consume.

Obstacles: Block movement and pathfinding. Do not damage.

Void tiles: Missing ground (Iteration 4) — block pathfinding and render as nothing. Used to chip corners and drop interior holes.

Actions

Melee attack: Hits all 8 adjacent tiles. Higher damage. Forces close engagement.

Ranged attack: Single target at Chebyshev distance 2–4. Lower damage. Safer positioning.

Mode toggle (Q): Switches between melee and ranged. Each mode upgrades independently.

End turn (Space): Only available after moving — prevents accidental skipped turns.

Combat

Two interactive elements interact here — see the dedicated section below. Enemy stats:

Enemy	HP	Speed	MoveRange	Damage	Behaviour
Soldier	30	1	2	8 melee	Path toward Shoki, attack adjacent
Sniper	20	2	2	5 ranged	Keep 2–4 tiles from Shoki
Heavy	60	0	1	12 melee	Slow approach, heavy hit

Companion stats:

Companion	HP	Speed	MoveRange	Damage	Niche
Drone	20	5	3	6 ranged	Glass cannon — keeps 2–4 tile range
Brawler	40	1	2	12 melee	Tank — soaks hits
Trickster	25	6	3	5 ranged	Acts twice per turn

Economy

No in-game currency. Progression is gated by post-wave upgrade choices (+25% melee OR +25% ranged) and companion recruitment at waves 3, 6, and 9. "Spending" is a one-way choice with no recovery.

Screen Flow

Screen Flow Chart

Screen Descriptions

- Main Menu — single "Start" button; minimal.
- Combat — main gameplay state; HUD overlays board.
- Upgrade Screen — post-wave; +25% melee or +25% ranged choice; shows heal amount.
- Companion Screen — at waves 3/6/9; three-card pick (Drone/Brawler/Trickster).
- Win / GameOver — restart-only end screens.

Game Options

Out of scope for the Week 6 prototype — no settings menu in the prototype; controls are fixed (left click + Q + Space).

Replaying and Saving

Out of scope for the Week 6 prototype — no save system; runs are session-only. Replayability comes from randomised room layouts, varied enemy compositions, and companion-pick branches.

Cheats and Easter Eggs

Out of scope for the Week 6 prototype — none planned.

Story, Setting and Character

Story and Narrative

Shoki crash-landed on Earth and government forces have surrounded the wreckage. To escape, he must hold his ground across 10 waves while repairing the ship's drive between rounds. Light pulp sci-fi tone; underdog framing — the player is the alien, the antagonists are very-grounded humans.

Back story

Shoki is a routine surveyor from a peaceful civilisation. A meteoroid collision forced an emergency landing on Earth — drawing the attention of a government rapid-response unit that has standing orders to capture any non-terrestrial life. The 10 waves represent successive deployments converging on the crash site as Shoki repairs his drive.

Plot Elements

- Inciting incident: the crash (begins before play)
- Rising action: waves 1–6 escalate; companions join at 3 and 6
- Climax: wave 9 introduces the third companion pick and the hardest pre-finale composition
- Resolution: wave 10 finale; on win, Shoki escapes; on loss, captured

Game Progression

License Considerations

Out of scope for the Week 6 prototype — student project; no third-party licensed content used. Sprites are original Aseprite work.

Cut Scenes

Out of scope for the Week 6 prototype — none — the 10-wave countdown carries the entire narrative arc.

Game World

General look and feel of world

Industrial crash-site debris on an open Earth plain. Tile palette uses warm tans (default ground) and cool purples (alternate) for visual rhythm. Traps are framed as crash-site hazards: live wires (spike), debris pits (pit), spilled coolant (slow zone).

Area #1

The crash site — the only area in the game. An 8×8 isometric room with procedurally varied silhouettes (corner chips, interior holes from Iteration 4) and randomized trap/obstacle placement.

Characters

Done: shoki.png, drone.png, sniper.png, heavy.png. Planned: soldier.png, brawler.png, trickster.png (current build uses tinted procedural squares as fallbacks).

Character #1

Shoki — the alien protagonist. HP 100, moveRange 2, melee 15 / ranged 8 base damage. Versatile (melee/ranged toggle), unremarkable mobility. His advantage is the player's ability to read the board — never raw stats.

Back story

Objectives

Interface

Visual System

The board communicates every relevant state without text. The visual gamestate decoder:

Visual cue	Meaning
Blue diamond on tile top	Valid move target this turn
Orange diamond on tile top	Valid attack target
Yellow pulsing diamond	Enemy will attack this tile next turn
Grey ghost diamond	Enemy will move to this tile next turn
Floating bar (green → red)	HP fraction of an enemy or companion
Particle burst	Hit landed — yellow = player attacks; red = player takes damage
Missing ground tile	Void — impassable, falls outside the room silhouette

HUD - What controls

- Top-left: Shoki HP bar and numeric value
- Top-right: Wave counter + companion roster (each with HP)
- Bottom-left: Damage multipliers (melee × ranged ×)
- Bottom-right: Current attack mode (Melee/Ranged) with [Q] hint
- Top-center: Controls reminder + telegraph legend

Menus

Minimal — Main Menu (single Start), Upgrade Screen (2 cards), Companion Screen (3 cards), Win/GameOver (restart). All built via OnGUI for prototype speed.

Rendering System

Unity Universal Render Pipeline (URP), 2D renderer. Sprites are point-filtered. Sort order is Y-based with explicit ranges: tiles -100..-1, highlights -50, entities 100+.

Camera

Single orthographic camera positioned at (0, 4, -10), orthographicSize = 6. Fixed framing of the 8×8 grid; no panning or zoom. Background colour is a deep purple to match the sci-fi tone.

Lighting Models

Out of scope for the Week 6 prototype — no lighting — sprites use their baked-in colour.

Control System

Input	Action
Left click	Move to highlighted tile, then attack on a second click
Q	Toggle Melee / Ranged attack mode
Space	End turn (only available after moving)

Audio

Music

Out of scope for the Week 6 prototype — planned post-prototype.

Out of scope for the Week 6 prototype — audio is out of scope for the Week 6 prototype.

Sound Effects

Out of scope for the Week 6 prototype — planned post-prototype. Visual juiciness (hit-burst particles, HP bars, telegraph pulses) substitutes for the feedback load.

Help System

On-screen HUD reminder always shows current controls. The telegraph legend appears top-centre explaining the yellow tile meaning. No tutorial; first wave is the onboarding (2 Soldiers, no traps elsewhere).

Artificial Intelligence

Opponent AI

At the start of every round each enemy runs ComputePlan() — pick the nearest player-aligned target, determine move destination, determine attack tile (if in range after the planned move). The plan is rendered as telegraph overlays (yellow for attack target, grey for move target). On the enemy's turn they COMMIT to the telegraphed plan, even if Shoki has moved. This gives the player a fair information advantage they can act on.

Enemy AI – Villains and Monsters

Type	Decision logic
Soldier	Pathfind toward Shoki via BFS. Plan attack if adjacent after move (Chebyshev ≤ 1).
Sniper	Maintain Manhattan distance 2–4. Retreat if too close, approach if too far. Plan ranged attack if in range after move.
Heavy	Same as Soldier but moveRange = 1 — slower threat, harder to dodge once close.

Non-combat Characters

Out of scope for the Week 6 prototype — none — every entity is either Shoki, an enemy, or a companion. All combatants.

Friendly Characters

- Drone — pathfind to keep Manhattan distance 2–4 from nearest enemy, ranged attack
- Brawler — pathfind to nearest enemy, melee attack adjacent
- Trickster — executes a Drone-like turn TWICE per round

Support AI

Companion AI runs the same plan/execute pattern as enemies but targets enemies instead of Shoki. Companions are persisted across waves via DontDestroyOnLoad and re-registered on each new room.

Player and Collision Detection

Tile-based — no continuous collision. Entity.MoveTo() sets occupancy on arrival and invokes TrapSystem.ResolveTrap. Two entities cannot occupy the same tile; BFS pathfinding respects occupancy.

Pathfinding

4-direction BFS in GridManager.FindPath. Treats Obstacle and Void tiles as blocked. GridManager.GetReachableTiles uses depth-limited BFS for the player's move-highlight system so the highlights match actual reachability (Iteration 7).

Technology

Target Hardware

Desktop — Linux (primary development target) and Windows. Mouse + keyboard input; no controller support in the prototype.

Development hardware and software

- Unity 6 (6000.3.2f1) — IDE and runtime
- VS Code / Rider — C# editing
- Aseprite — sprite authoring
- Git — version control

Development procedures and standards

Game Engine

Unity 6 with Universal Render Pipeline (URP) configured for 2D. No third-party engine plugins. Procedural placeholder graphics generate missing sprites at runtime (Aseprite PNGs override them when available).

Network

Single-player only. No networking; no multiplayer planned for the prototype.

Scripting Language

C# only. No visual scripting, no external state machines. Coroutines drive turn execution and animation.

etc.

Game Art

Concept Art

Concept passes are integrated directly into Aseprite — there is no separate concept-art stage. Sprite iterations live as Aseprite layers/versions on disk.

Style Guides

16-pixel base unit, low palette count, no anti-aliasing, no outlines except for cube tile depth. Iso projection is 2:1 (diamond width:height). Sprite pivots: tiles at (0.5, 0.75) so the diamond top aligns with the grid position; entities at (0.5, 0) so they stand on the tile.

Characters

Environments

Two tile sprites in rotation: ground.png (tan) and ground_alt.png (purple). RoomGenerator carves void tiles per wave for silhouette variation.

Equipment

None — combat actions are character-intrinsic. No weapon items to pick up.

Cut scenes

Miscellaneous

Management

Detailed Schedule

- Weeks 1–2: Concept lock + GDD first draft (Iteration 1: top-down → isometric pivot)
- Week 3: Core systems — grid, turns, BFS pathfinding, traps (Iteration 2: telegraphs + companion design)
- Week 4: Enemy AI, companion AI, combat resolution (Iteration 3: damage rebalance)
- Week 5: Visual juiciness — HP bars, hit particles, smooth walking (Iterations 4–6)
- Week 6: Stability + intermediate playtest + presentation (Iteration 7)

Budget

Out of scope for the Week 6 prototype — educational project; no monetary budget. All tools are free or already owned.

Risk Analysis

- Scope creep — mitigated by hard 10-wave cap and "out of scope" notes on optional template sections
- Art bottleneck — addressed by procedural fallback sprites so gameplay can be tuned without finished art
- Difficulty tuning — addressed by Iteration 3 damage rebalance and the wave progression curve table
- Turn-order desyncs — addressed by Iteration 7 stability fix and code comments to prevent regression

Localization Plan

Out of scope for the Week 6 prototype — English only.

Test Plan

Iterative playtest cycles (see Iteration History below) drive design changes. Each cycle observes specific failure modes (wave 2 deaths, repetitive rooms, unclear highlights, weightless hits, broken turn order) and ships a targeted fix.

Appendices

Asset List

Master list of all in-game assets, current and planned. Subsection tables below.

Art

Asset	Status	Notes
shoki.png	Done	32×32, bottom-center pivot
drone.png	Done	Cyan glass-cannon companion
sniper.png	Done	Camo-green enemy
heavy.png	Done	Navy steel enemy
soldier.png	Planned	Grey gunmetal grunt — procedural fallback in current build
brawler.png	Planned	Amber mech-suit companion
trickster.png	Planned	Magenta saboteur companion
ground.png	Done	Tan isometric cube tile
ground_alt.png	Done	Purple variant for visual rhythm
VFX / animations	Planned	Sprite animations to hook into the WalkToTileSmooth coroutine

Model and Texture List

Animation List

Effects List

Interface Art List

Cut scene List

Sound

Out of scope for the Week 6 prototype — audio is out of scope for the Week 6 prototype.

Environmental Sounds

Weapon Sounds

Interface Sounds

Music

Ambient

“Action”

Victory

Defeat

Actor #1 lines

Two Interactive Elements

The DDG brief requires at least two interactive elements (beyond the player character) that follow the sensory-input → processing → action pattern, and that can interact with each other. Shoki's Adventure meets this with two systems that explicitly form a feedback loop.

Element 1 — Enemy AI. See: queries TurnManager for player-faction entities; processing: ComputePlan() determines move destination and attack tile; action: commits to telegraphed plan on the enemy's turn.

Element 2 — Environmental Traps. Collision: Entity.MoveTo invokes TrapSystem.ResolveTrap on arrival; processing: switch on trap type; action: deals damage, applies status, or rewrites tile state.

Interaction: Enemies COMMIT to telegraphed plans and their pathfinder treats trap tiles as walkable. The player can bait them onto pits — e.g., a Heavy telegraphed to walk south through a Pit will step in and die. This is the system-on-system interaction the rubric asks for.

Playability, Accessibility & Juiciness

Playability: every legal action is visualised before commitment (move + attack highlights); no hidden information (telegraphs + HP bars); minimal controls (click + Q + Space).

Accessibility: one-handed input; high-contrast UI; HP encoded with colour AND length so colour-blind players parse correctly; no reflex requirements (turn-based).

Juiciness: hit-burst particles on every damage event; floating HP bars that shrink and colour-shift; smooth tile-to-tile walking; pulsing yellow telegraph tiles; post-wave heal feedback.

Playtest Feedback Log

DDG-required documentation of what each playtest cycle observed and what was changed in response.

Cycle	Observation	Decision
Concept review	30×30 top-down grid was too large to read at a glance.	→ Iteration 1: pivot to isometric 8×8, 2-tile cardinal movement.
Design review	Pokémon-type counters added memorisation tax without making the puzzle harder.	→ Iteration 2: drop type counters; add Companion system + telegraphs.
Playtest 1	"I died on wave 2 every time." Worst-case round = 63 damage.	→ Iteration 3: enemy damage cut ~45%; heal raised 50% → 75%.
Playtest 2	"All rooms look the same."	→ Iteration 4: per-wave Void tile carving.
Playtest 3	"I can't tell which tiles are valid attack targets."	→ Iteration 5a: top-face-only highlights.
Playtest 3	"How much HP does this guy have?"	→ Iteration 5b: floating HP bars.

Playtest 3	"Hits feel weightless."	→ Iteration 5c: hit-burst particles.
Playtest 4	"Shoki just teleports."	→ Iteration 6: smooth path-walking.
Stability	"Clicked a diagonal blue tile, Shoki stopped one short."	→ Iteration 7a: BFS-reachable highlights.
Stability	"Game froze when an enemy walked into a pit."	→ Iteration 7b: UnregisterEntity index fix.

Iteration History

Detailed reasoning for each of the seven iterations the prototype has gone through.

Iteration 1 — Top-down 30×30 → Isometric 8×8. Trigger: concept review. Why: 30×30 was unreadable; 8×8 keeps decisions tight. Required rewriting all spatial systems.

Iteration 2 — Drop type counters; add Companions + Telegraphs. Trigger: design review. Why: type counters added cognitive load without depth. Companions add permadeath stakes; telegraphs eliminate hidden information.

Iteration 3 — Damage rebalance. Trigger: playtest 1. Why: wave-6 worst case 63 dmg = lethal in 2 rounds. New numbers cap at 33 dmg/round, recoverable with 75% heal.

Iteration 4 — Void tile carving. Trigger: playtest 2 (room fatigue). Why: static 8×8 rectangles felt repetitive. Per-wave silhouette variation changes tactical character.

Iteration 5 — Juiciness pass (highlights, HP bars, particles). Trigger: playtest 3 feedback. Why: directly addresses Playability/Accessibility/Juiciness rubric. Combat reads as combat now.

Iteration 6 — Smooth path-walking. Trigger: animation prep + trap-timing readability. Why: teleport broke trap timing; smooth walking creates an animation hook.

Iteration 7 — Stability + reachability fixes. Trigger: stability playtest. Why: BFS-reachable highlights stopped "corner stop short" bug; UnregisterEntity index fix stopped turn-order desync when an entity died mid-turn.

Lecture Learnings Coverage

DDG requires at least 5 of 9 lecture learnings to be addressed. This GDD covers 8.

Learning	Where it lives in this GDD
Core loop diagram (micro/macro/meta)	Game Flow section
Intended fun & engagement	Look and Feel → LeBlanc fun type
GDD (mandatory)	This entire document
Multiple prototype iterations	Playtest Feedback Log + Iteration History (7 entries)
Interesting choices & their evolution	Actions + Iteration 2 (companion system)
Balancing work	Iteration 3 + Game Progression wave-curve table
Randomness & changes	Game Progression + Area #1
Narrative aspects	Story and Narrative + Back story
Visual gamestate	Visual System decoder table
Design patterns	Singleton, Observer, State machine, Component composition, Coroutines — used throughout the codebase